

UNIVERSITY OF KARACHI

UBIT — DEPARTMENT OF COMPUTER SCIENCE

PROJECT REPORT

HealthNest

Doctor Appointment Booking SaaS Platform

A Collaborative Project Submitted in Partial Fulfilment of the Requirements for the Course

Course Title	Internet Application Development
Course Code	BSCS-633
Supervisor	Sir Zaeem Tariq

GROUP MEMBERS

Name	Seat Number
Haseeb Uddin	B2210006053
Daniyal Ahmed	B21110006024

Session 2026

Submitted to the Department of Computer Science, UBIT

Table of Contents

Abstract3

1. Introduction3

 1.1 Background 3

 1.2 Problem Statement..... 3

 1.3 Objectives 3

 1.4 Scope of the Project..... 3

2. System Overview4

 2.1 Why This Idea..... 4

 2.2 User Roles 4

3. Technology Stack4

4. System Architecture5

 4.1 Communication Flow 5

5. Database Design6

 5.1 Entity Summary..... 6

6. Use Case Diagram7

7. Key Features and Modules8

8. Security Implementation8

9. Deployment.....9

10. Testing.....9

11. Conclusion and Future Enhancements10

12. References.....10

Abstract

Scheduling a doctor's appointment still commonly requires a phone call or an in-person visit to a clinic, a process that is time-consuming for patients and difficult to manage for healthcare providers. HealthNest is a full-stack, role-based healthcare platform designed to resolve this gap by allowing patients to search for doctors, view live slot availability, and book appointments online, while doctors manage their schedules and administrators oversee the entire platform from a single dashboard. The system is built using React (Vite) on the frontend and ASP.NET Core Web API on the backend, backed by a PostgreSQL database and real-time communication through SignalR. It further integrates an AI-assisted symptom checker (Google Gemini) to help patients identify the right specialist, and a live queue module that lets patients track their position in real time. The platform was containerised with Docker and deployed using Render (backend) and Vercel (frontend), demonstrating a complete, production-style deployment pipeline suitable for a modern SaaS application.

1. Introduction

1.1 Background

Healthcare access in many clinics and hospitals is still managed manually — patients call the front desk, wait on hold, or physically visit the facility just to check whether a doctor is available. This creates friction for patients and administrative overhead for staff, and it leaves no easy way for a patient to know how long they may have to wait once they arrive. As internet-based services have become the default way people book everything from flights to restaurant tables, there is a clear opportunity to bring the same convenience to healthcare scheduling.

1.2 Problem Statement

There is no lightweight, easy-to-adopt system that lets a small clinic or a group of independent doctors manage appointments, availability, and patient queues online, while also giving patients a simple way to find the right specialist and track their appointment status in real time.

1.3 Objectives

- Design and develop a role-based web platform for Patients, Doctors, and Admins.
- Allow patients to search doctors by specialization and book available time slots online.
- Give doctors full control over their profile, availability, and appointment status.
- Provide administrators with a central dashboard to manage doctors, patients, appointments, and analytics.
- Implement a real-time queue system so patients can track their position without needing to be physically present.
- Integrate an AI-based symptom checker to guide patients toward the correct specialization.
- Deploy the system using modern, industry-standard cloud infrastructure.

1.4 Scope of the Project

The scope of HealthNest covers appointment booking and clinic-workflow management for three user roles. It does not currently cover payment processing, video consultations, or integration with external hospital information systems (HIS); these are identified as potential future enhancements in the concluding section of this report.

2. System Overview

HealthNest is a Software-as-a-Service (SaaS) style doctor appointment booking system. Patients can search for doctors, check their available time slots, and book an appointment online. Doctors can manage their profile, set their available slots, and track their appointments. Administrators can manage the whole platform — doctors, patients, and bookings — from one dashboard.

2.1 Why This Idea

Booking a doctor's appointment usually still means calling the clinic or waiting in person. The team wanted to build something that solves that everyday problem: a simple online system where a patient can find the right doctor and book a slot in a few clicks, while the doctor manages their day without paperwork.

2.2 User Roles

- **Patient** — registers, searches for doctors, books appointments, checks in to the live queue, and uses the AI symptom checker.
- **Doctor** — manages their profile and available time slots, and confirms, completes, or cancels appointments from their dashboard.
- **Admin** — manages all doctors and patients, oversees every appointment, and reviews platform-wide analytics.

3. Technology Stack

The platform is built entirely on free-tier, modern web technologies, chosen for their strong ecosystem support, real-time capability, and ease of deployment.

Layer	Technology	Purpose
Frontend	React 18 + Vite	Single-page application UI for all three roles
Routing	React Router v6	Client-side routing and protected role-based routes
Styling / UX	Custom CSS + Framer Motion	Page transitions, animations, light/dark theming
Charts	Recharts	Admin analytics dashboards

Backend	ASP.NET Core (.NET 10) Web API	REST API, business logic, authentication
ORM	Entity Framework Core	Object-relational mapping and migrations
Database	PostgreSQL	Persistent relational data storage
Real-time	SignalR	Live queue updates and instant booking notifications
Auth	JWT Bearer + BCrypt.Net	Token-based authentication and password hashing
AI Service	Google Gemini API	AI-assisted symptom analysis with safe fallback
Containerisation	Docker	Portable backend deployment image
Hosting	Render (API) + Vercel (Frontend)	Cloud deployment of backend and frontend

4. System Architecture

HealthNest follows a layered client-server architecture. The React single-page application communicates with the ASP.NET Core Web API over HTTPS for standard REST operations, and maintains a persistent WebSocket connection to a SignalR hub for real-time events such as queue updates and booking notifications. The API layer talks to a PostgreSQL database through Entity Framework Core, and calls out to the Google Gemini API for AI-assisted symptom analysis. The frontend is hosted on Vercel and the backend is containerised with Docker and hosted on Render.

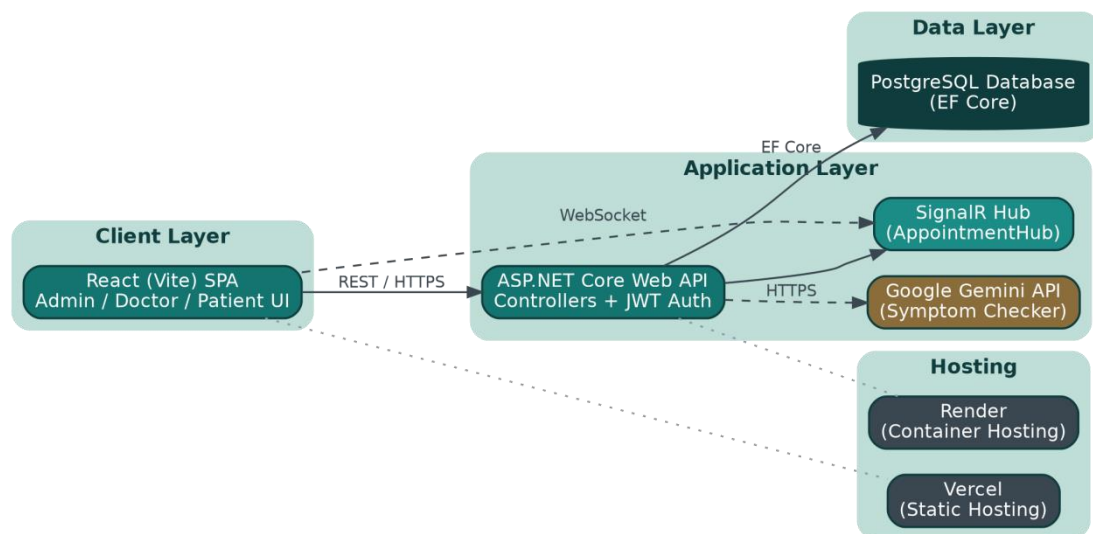


Figure 4.1 — HealthNest System Architecture

4.1 Communication Flow

- The browser sends authenticated REST requests to the API for booking, profile, and admin operations.
- The browser also opens a WebSocket connection to the SignalR hub for live queue and notification events.
- The API validates each request's JWT, applies role-based authorization, and reads/writes data via EF Core.
- For the symptom checker, the API forwards the patient's description to the Gemini API and returns a recommended specialization, falling back to a rule-based response if no API key is configured.

5. Database Design

The data model is intentionally kept lean, with four core entities that map directly onto the platform's booking workflow: Users (who can be a Patient, Doctor, or Admin), Doctor Profiles (specialization-specific data owned by a Doctor user), Time Slots (published availability), and Appointments (a booking made against a specific slot).

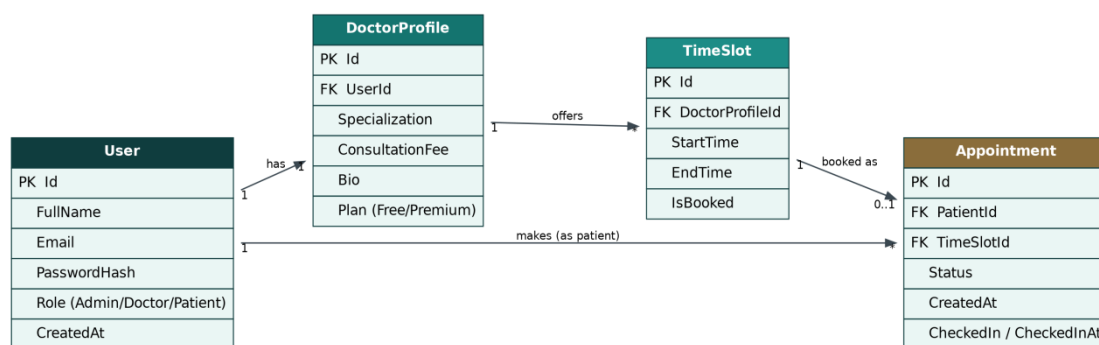


Figure 5.1 — Entity-Relationship Diagram

5.1 Entity Summary

- **User** — stores account credentials and role (Admin, Doctor, or Patient). A Doctor-role user owns exactly one DoctorProfile.
- **DoctorProfile** — holds specialization, consultation fee, bio, and subscription plan (Free/Premium), and owns many TimeSlots.
- **TimeSlot** — represents a bookable time window for a doctor, flagged as booked once an appointment is placed against it.
- **Appointment** — links a Patient (User) to a TimeSlot, and tracks status (Pending, Confirmed, Completed, Cancelled) plus live-queue check-in state.

6. Use Case Diagram

The diagram below summarises the primary interactions available to each of the three actors in the system: Patient, Doctor, and Admin. All three roles share a common Register/Login use case, after which the system routes each user to a dedicated, role-specific dashboard.

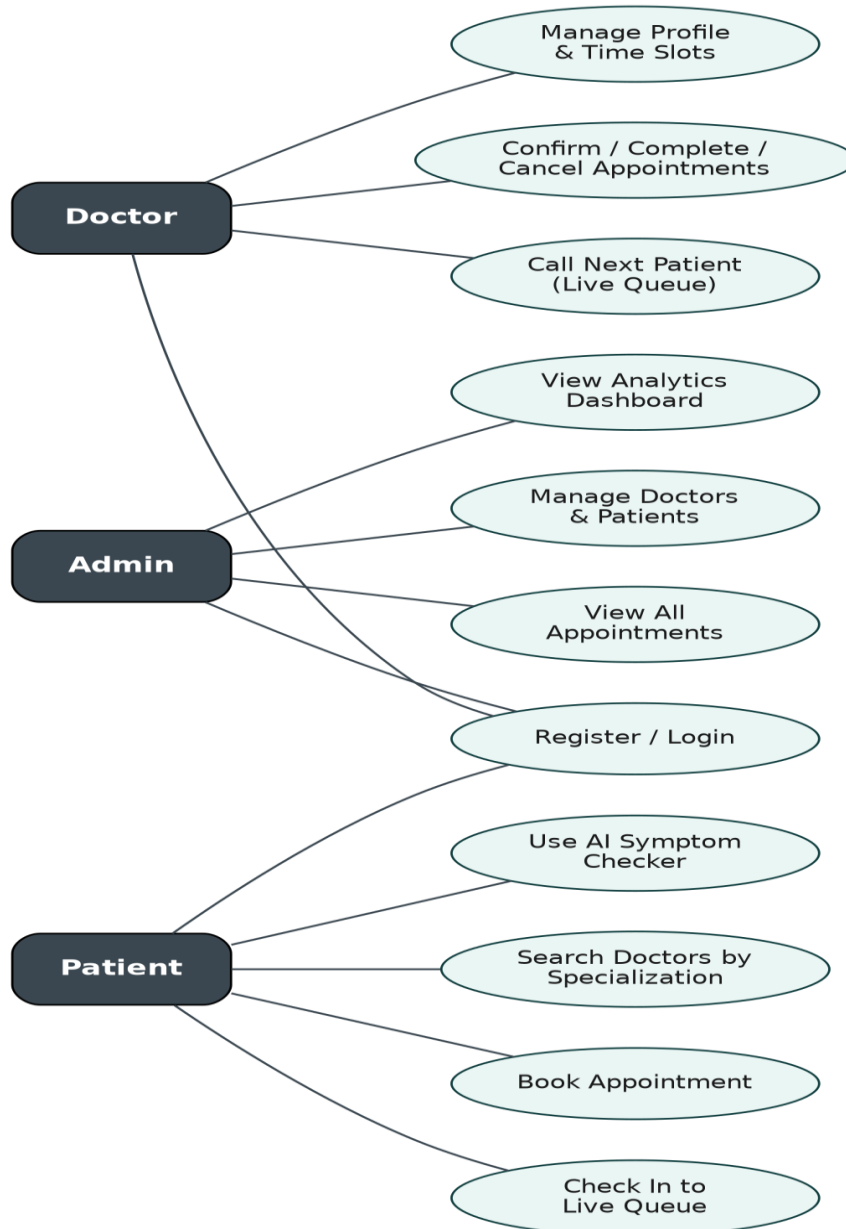


Figure 6.1 — Use Case Diagram

7. Key Features and Modules

7.1 Patient Module

- Search and browse doctors by specialization.
- View a doctor's live slot availability and book an appointment in a few clicks.
- Track appointment status (Pending, Confirmed, Completed, Cancelled).
- Check in online and see live queue position update in real time.
- Describe symptoms in plain text and receive an AI-suggested specialization.

7.2 Doctor Module

- Manage personal profile: specialization, consultation fee, and bio.
- Create and publish available time slots.
- Confirm, complete, or cancel booked appointments.
- Run the live queue: call the next checked-in patient.

7.3 Admin Module

- Manage the full list of doctors and patients on the platform.
- View and oversee every appointment across all doctors.
- Review an analytics dashboard: booking summaries, daily trends, top doctors, and breakdown by specialization.

7.4 Real-Time Notifications and Live Queue

HealthNest uses SignalR to push instant updates to connected clients without requiring a page refresh. When a patient books a slot, the assigned doctor is notified immediately; when a patient checks in, their position in the queue is broadcast live to both the patient and the doctor's queue view.

7.5 AI Symptom Checker

A patient can describe their symptoms in natural language through a chat-style widget. The request is sent to the backend, which calls the Google Gemini API to analyse the description and recommend a specialization along with a short explanation. If no Gemini API key is configured, the system falls back to a safe, rule-based recommendation so the feature still functions in a degraded mode.

8. Security Implementation

- **Password Hashing** — user passwords are never stored in plain text; BCrypt.Net is used to hash and verify credentials.

- **JWT Authentication** — every authenticated request carries a signed JSON Web Token validated for issuer, audience, lifetime, and signing key.
- **Role-Based Authorization** — API endpoints and frontend routes are gated by role (Admin, Doctor, Patient), enforced on both client and server.
- **CORS Policy** — only explicitly allowed origins (the deployed frontend URL) may call the API.
- **Secure Real-Time Auth** — since browsers cannot attach custom headers to a WebSocket handshake, the JWT is passed as a query parameter and validated server-side before a SignalR connection is accepted.

9. Deployment

The backend and frontend are deployed independently, mirroring how modern SaaS products separate API and client hosting.

- **Backend** — containerised with Docker and deployed to Render, with a managed PostgreSQL instance provisioned alongside it. Database migrations are applied automatically on startup.
- **Frontend** — built with Vite and deployed as a static site on Vercel, configured to rewrite all routes to index.html for client-side routing.
- **Configuration** — environment variables (API URL, SignalR hub URL, allowed CORS origins, JWT secret, Gemini API key) are managed separately per environment and never committed to source control.

On the free hosting tier, the backend service sleeps after 15 minutes of inactivity; the first request afterward can take 30–50 seconds while the service wakes up. This is an expected characteristic of free-tier hosting rather than a defect in the application.

10. Testing

The application was tested manually across all three roles to confirm that core workflows behave as expected before deployment.

Test Scenario	Expected Result
Patient registers and logs in	Account created; JWT issued; redirected to Patient dashboard
Patient books an available time slot	Slot marked as booked; appointment created with Pending status
Doctor confirms a pending appointment	Status updates to Confirmed; patient view reflects change in real time
Patient checks in for an appointment	Patient appears in the doctor's live queue; position updates via SignalR

Admin deletes a doctor profile	Doctor and associated data removed from admin listings
Symptom checker with no Gemini key configured	System returns a safe fallback recommendation instead of failing
Unauthorized role attempts to access a protected route	Access denied / redirected, per role-based route protection

11. Conclusion and Future Enhancements

11.1 Conclusion

HealthNest successfully demonstrates a complete, role-based healthcare booking platform built with a modern web stack. It brings together authentication, real-time communication, relational data modelling, and third-party AI integration into a single cohesive product, and shows a full path from local development to a live, publicly deployed application. The project met its core objectives: patients can find and book doctors online, doctors can manage their schedule and queue, and admins can oversee the platform from a single dashboard.

11.2 Future Enhancements

- Online payment integration for consultation fees.
- Video consultation support for remote appointments.
- SMS/email reminders for upcoming appointments.
- Patient medical history and prescription records.
- Mobile application (React Native) sharing the same backend API.